

Greedy Weighted Graph Spanners

Implementation and Experimental Evaluation

Mini-Project on Embeddings of Graphs

Ben-Gurion University of the Negev

By: Alon Ganot & Tomer Akdam

Date: June 2026

Source paper: I. Althofer, G. Das, D. Dobkin, D. Joseph, and J. Soares, "On Sparse Spanners of Weighted Graphs", Discrete & Computational Geometry, 1993.

Abstract. This report presents an implementation and empirical evaluation of the greedy weighted graph spanner algorithm of Althofer et al. The algorithm processes weighted edges in nondecreasing order and keeps an edge only when the current spanner distance is too long. The project implements the construction in Java, verifies the stretch and MST-related invariants, and evaluates the tradeoff between sparsity, total weight, and observed stretch across paths, cycles, trees, complete graphs, random connected graphs, planar grids, and theta-style hard examples. Across 270 full experimental runs, no stretch violations were found. Increasing k from 1 to 5 raised average edge reduction from 24.24% to 48.56%, while average lightness dropped from 3.91 to 1.01 times MST weight.

Table of Contents

1. Project Requirements and Scope

2. Background and Paper Summary

3. Implemented Algorithm

4. Software Design

5. Experimental Methodology

6. Experimental Results

7. Discussion

8. Conclusions and Future Work

References

Appendix A. Reproducibility Commands

1. Project Requirements and Scope

The syllabus defines the mini-project goal as reading a paper in low-distortion graph embeddings, implementing one algorithm from the paper, running experiments, and summarizing the results in a final report. The selected paper is Althofer, Das, Dobkin, Joseph, and Soares, "On Sparse Spanners of Weighted Graphs". The implemented algorithm is the paper's greedy SPANNER construction for weighted undirected graphs.

The project focuses on sparse subgraphs called spanners. Given a connected weighted graph G , the goal is to build a subgraph H on the same vertex set such that all pairwise distances in H are at most a controlled multiplicative factor larger than in G . The experiments therefore measure three quantities: distance preservation, edge sparsity, and total weight relative to a minimum spanning tree.

- Algorithmic requirement: implement the weighted greedy spanner construction.
- Experimental requirement: evaluate the algorithm on several graph families and parameters.
- Report requirement: explain the theory, implementation decisions, measured results, and limitations.

2. Background and Paper Summary

2.1 Spanner definition

Let $G=(V,E,w)$ be an undirected graph with positive edge weights. For vertices u,v in V , let $d_G(u,v)$ be the shortest-path distance in G . A subgraph $H=(V,E_H,w)$ is an r -spanner of G if:

$$d_H(u,v) \leq r \cdot d_G(u,v) \quad \text{for every } u,v \text{ in } V.$$

Because H is a subgraph of G , distances in H cannot be shorter than distances in G . The only possible distortion is that paths become longer after edges are removed. A good spanner removes many edges while keeping r small.

2.2 Sparsity and lightness

The paper distinguishes between two types of sparsity. Size sparsity concerns the number of edges in the spanner, $|E_H|$. Weight sparsity concerns the total edge weight $w(H)$. The minimum spanning tree $MST(G)$ is the natural baseline for weight because it is the lightest connected spanning subgraph, but an MST alone can have poor stretch on graphs such as cycles.

2.3 Main theoretical guarantees used in the project

The paper proves that the greedy construction produces sparse spanners for arbitrary weighted graphs. In the paper notation, running $SPANNER(G,r)$ with $r=2t+1$ yields a $(2t+1)$ -spanner. The project uses the more common implementation parameter k and sets $r=2k-1$. Therefore, for the theorems that use t , the mapping is $t=k-1$.

case	stretch	size comparison	use in report
General weighted graph	stretch $2k-1$	$O(n^{1+1/k})$ edges	paper gives a polynomially constructible sparse spanner; project reports $ E_H /n^{1+1/k}$
Planar weighted graph, $k \geq 2$	stretch $2k-1$	$(n-1)(1+1/(k-1))$ edges	paper gives a sharper planar bound; project computes this ratio for planar families
Weight baseline	same stretch	MST contained in H	measured as $w(H)/w(MST(G))$

Table 1. Mapping between the paper notation and the project notation.

2.4 Key proof ideas from the paper

1. Shortest-path replacement proves stretch. If an edge $e=(u,v)$ is rejected, the current spanner already contains a u - v path of length at most $r \cdot w(e)$. Replacing every rejected edge on a shortest G -path by its stored short path gives the r -stretch guarantee.
2. No short cycles proves edge sparsity. If the output contained a short cycle, the last edge of that cycle processed by the greedy order would have been rejected. Therefore the output has large girth, which implies few edges by extremal graph bounds.
3. MST containment follows from Kruskal-like behavior. When an edge connects two disconnected components in the current partial spanner, its current distance is infinite and it is accepted. This parallels Kruskal's algorithm and implies that the output contains at least one MST of the input graph.
4. Planar graphs admit stronger bounds. In planar graphs, face-size and Euler-formula arguments give a linear edge bound with a better multiplicative constant.

3. Implemented Algorithm

The implementation follows the greedy SPANNER algorithm directly. The input is a weighted undirected graph with positive finite edge weights. The output is a new graph H containing all vertices and only the selected edges.

```
GreedyWeightedSpanner( $G, k$ ):  
     $r = 2k - 1$   
    sort all edges by nondecreasing weight  
     $H =$  graph with  $V(G)$  and no edges  
    for each edge  $e=(u,v)$  in sorted order:  
        compute current shortest-path distance  $d_H(u,v)$   
        if  $d_H(u,v) > r * w(e)$ :  
            add  $e$  to  $H$  with its original weight  
    return  $H$ 
```

The strict comparison is important: the edge is added only when the current path is longer than the allowed threshold. If the current path is equal to the threshold, the edge is not needed for the requested stretch.

3.1 Behavior at $k=1$

For $k=1$, the stretch bound is 1. This does not necessarily mean that every original edge must remain. The output must preserve all shortest-path distances exactly. A heavy direct edge can still be removed if an equal or shorter path already exists through lighter edges. This explains why weighted complete graphs can already lose many nonessential edges at $k=1$, while complete unit graphs keep all edges at $k=1$ because every direct edge is itself a unit shortest path.

4. Software Design

The implementation is written in Java and uses JGraphT for weighted graph storage, Dijkstra shortest paths, and Kruskal MST computations. The design separates graph construction, spanner construction, metric computation, experiments, tests, and plotting.

component	role
GreedyWeightedSpanner.java	Core greedy construction; validates k and positive weights; computes Dijkstra on the current spanner for each candidate edge.
GraphMetrics.java	Computes total weight, MST weight, maximum observed stretch, stretch violations, and tie-safe MST containment.
GraphFactory.java	Creates deterministic graph families: paths, cycles, random trees, complete graphs, random connected graphs, grids, and theta-style examples.
ExperimentRunner.java	Runs quick and full experiment suites for $k=1..5$ and writes CSV rows.
SanityTests.java	Checks correctness properties before running the experimental pipeline.
make_plots.py	Creates derived metrics, tables, quantitative plots, and visual graph examples.

Table 2. Main project components.

4.1 Correctness checks

The current project archive was compiled and the Java sanity tests were run under OpenJDK 21.0.10. All six tests passed:

```
PASS triangle heavy edge skipped
PASS tree unchanged
PASS k=1 exact distances
PASS MST containment
PASS stretch bound on deterministic random graphs
PASS invalid weights rejected
PASS all sanity tests (6/6)
```

A practical implementation detail is that MST containment is checked in a tie-safe way. If weights are equal, several MSTs may exist, so the project checks whether the spanner contains some MST of the original graph by comparing MST weights, instead of requiring a particular arbitrary MST returned by a library.

5. Experimental Methodology

The full experiment suite evaluates $k=1,2,3,4,5$ on 54 deterministic scenarios, giving 270 rows. The scenario set intentionally mixes trivial cases, dense cases, planar cases, random cases, wide-weight cases, and constructed hard examples.

family	purpose
paths and trees	Sparse baselines; no cycle edges should be removable because the graph is already minimally connected.
cycles and cycle-threshold examples	Show exactly when the final cycle edge can be skipped under a given stretch.
complete weighted graphs	Dense inputs where greedy sparsification is expected to be strongest.
complete unit graphs	Clean contrast between $k=1$ exact preservation and $k \geq 2$ tree-like output.
random connected graphs	Sparse, medium, and dense random benchmarks for average-case behavior.
grid planar graphs	Structured planar weighted examples connected to the paper's planar theorem.
theta-style lower-bound graphs	Constructed examples that resist sparsification at the matching stretch.
wide-weight-range graphs	Stress behavior when weights vary over a larger range.

Table 3. Experiment graph families and their role.

5.1 Metrics

- Edge reduction: $100 \cdot (1 - |E_H|/|E_G|)$. Higher is sparser.
- Lightness: $w(H)/w(\text{MST}(G))$. A value close to 1 means the spanner is nearly as light as an MST.
- Maximum observed stretch: max over pairs u,v of $d_H(u,v)/d_G(u,v)$.
- Stretch violations: number of pairs exceeding the allowed bound $2k-1$.
- General size-bound ratio: $|E_H|/n^{1/(1+1/k)}$, used as an empirical scale comparison for general graphs.
- Planar size-bound ratio for known planar scenarios and $k \geq 2$: $|E_H| / ((n-1)(1+1/(k-1)))$.

6. Experimental Results

The headline result is that the empirical behavior matches the theory: no stretch violations occurred in any of the 270 full experiment rows. As k increases, the algorithm allows longer replacement paths and therefore tends to keep fewer and lighter edges.

k	$2k-1$	avg $ E_H $	avg edge reduction %	avg $w(H)/w(\text{MST})$	max observed stretch	avg stretch utilization	avg runtime ms
1.000	1.000	147.5	24.24	3.909	1.000	1.000	2.370
2.000	3.000	78.61	43.50	1.224	3.000	0.711	1.389
3.000	5.000	71.93	46.40	1.065	5.000	0.645	0.981
4.000	7.000	70.04	47.89	1.024	7.000	0.604	1.000
5.000	9.000	69.20	48.56	1.009	9.000	0.561	0.852

Table 4. Summary by k over all 54 scenarios.

Sparsity and weight tradeoff as stretch increases

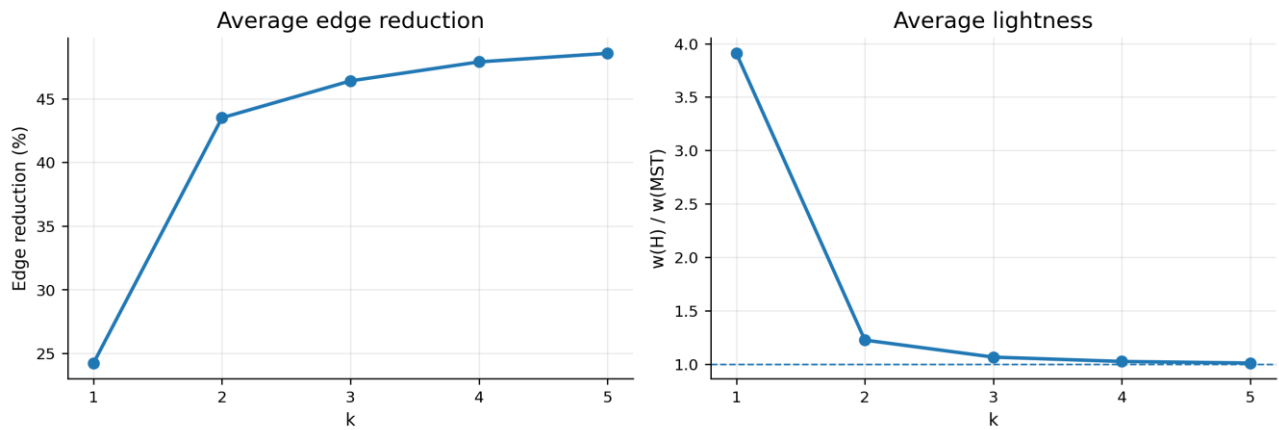


Figure 1. Average sparsity and lightness improve as k increases. The dashed line in the right panel marks MST weight.

6.1 Stretch validation

The maximum observed stretch reached the allowed bound in some constructed cases, but never exceeded it. This is a useful result: it shows that the test suite is not only passing easy cases where the observed stretch is far below the bound. Some examples hit the bound exactly, which exercises the edge-rejection threshold.

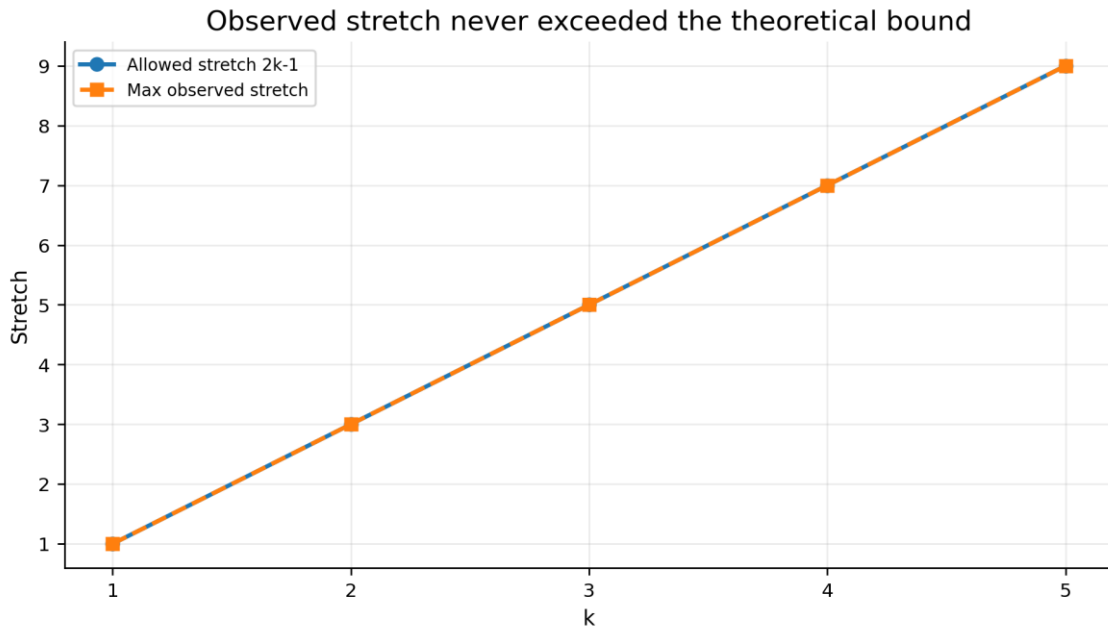


Figure 2. Maximum observed stretch compared with the theoretical bound $2k-1$.

6.2 Behavior by graph family

Dense graphs are the easiest to sparsify. Complete weighted graphs averaged about 92.05% edge reduction at $k=3$ while remaining close to MST weight. Complete unit graphs show a sharper transition: at $k=1$ all unit edges are required to preserve exact distances, while for $k \geq 2$ the greedy output collapses to $n-1$ edges in the tested cases. Paths and trees do not shrink, because removing any edge disconnects the graph. Theta-style lower-bound examples resist sparsification around their target stretch, which is exactly the behavior expected from the paper's lower-bound discussion.

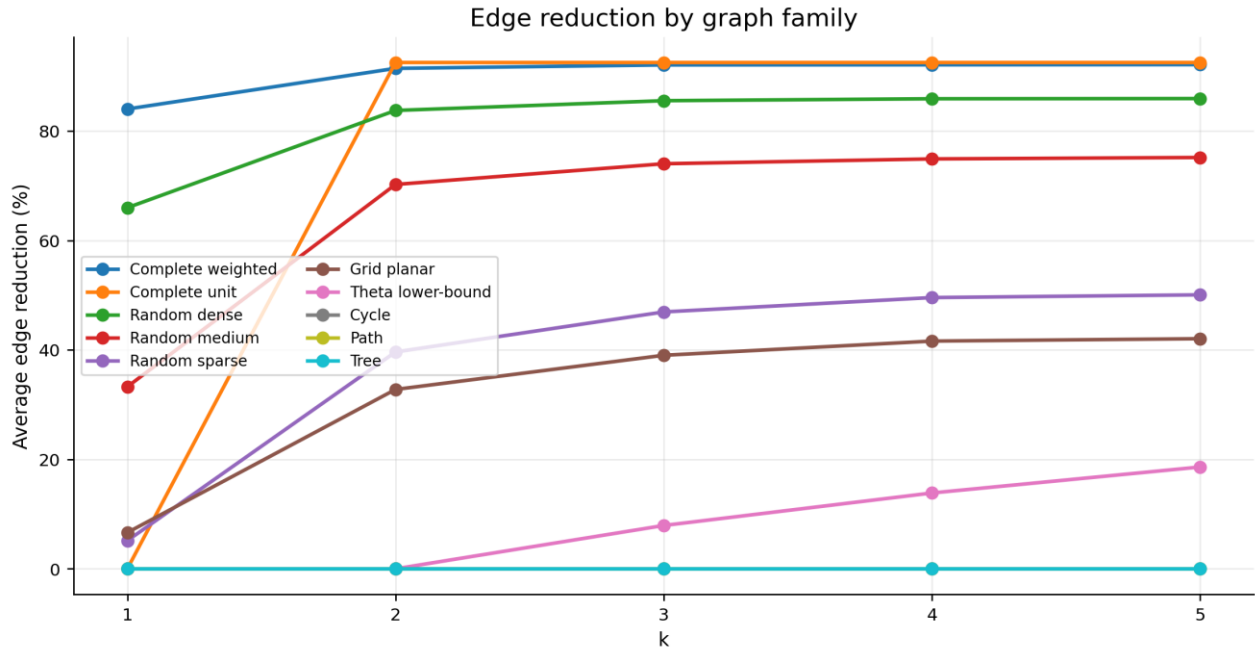


Figure 3. Average edge reduction by selected graph family.

family	scenarios	avg $ E_G $	avg $ E_H $	avg edge reduction %	avg $w(H)/w(MST)$	max stretch
Complete wtd	6	579.3	31.50	92.05	1.016	5.000
Complete unit	2	485.0	29.00	92.50	1.000	2.000
Cycle	4	85.00	85.00	0.000	1.017	1.000
Grid planar	3	185.3	110.0	39.01	1.145	5.000
Path	4	84.00	84.00	0.000	1.000	1.000
Random dense	4	489.0	71.25	85.52	1.045	4.900
Random medium	11	366.3	95.45	74.00	1.081	5.000
Random sparse	3	205.7	110.3	46.93	1.134	5.000
Theta hard	4	90.00	85.25	7.917	1.184	5.000
Tree	4	84.00	84.00	0.000	1.000	1.000

Table 5. Family highlights at $k=3$.

6.3 Planar benchmarks

For planar families and $k \geq 2$, the project computed the ratio to the paper's planar edge bound after applying the parameter mapping $t=k-1$. All tested planar families stayed below ratio 1. Grid graphs were nontrivial because the greedy spanner removed many edges while maintaining shortest-path approximation on a structured weighted planar graph.

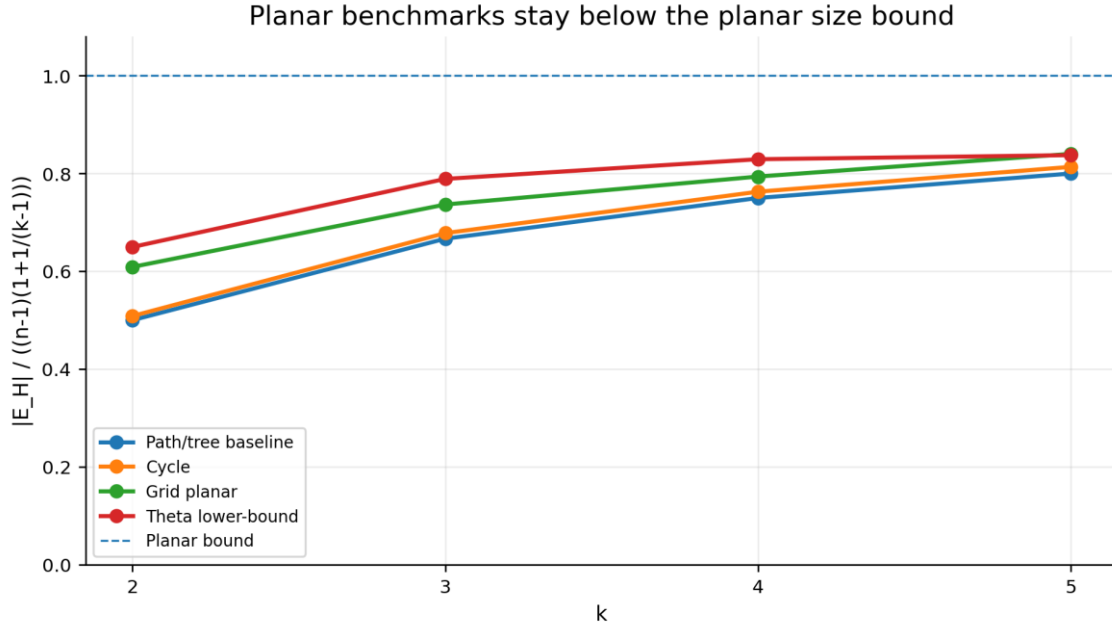


Figure 4. Ratio to the planar size bound for selected planar benchmark families.

6.4 Visual examples

The visual examples compare the original graph with the greedy spanner. Blue edges are a chosen MST contained in the spanner; red edges are additional spanner edges; pale gray edges are removed. These examples are useful for the presentation because they make the abstract distance/sparsity tradeoff visible.

example	k	n	E_G	E_H	edge reduction %	max stretch	bound	w(H)/w(MST)
complete n10	2	10	45	10	77.78	2.091	3.000	1.214
random n14	3	14	35	13	62.86	3.000	5.000	1.000
grid 5x5	3	25	40	24	40.00	4.500	5.000	1.000
theta K3	3	20	24	24	0.000	1.000	5.000	1.263
cycle n7	4	7	7	6	14.29	6.000	7.000	1.000
unit K20	2	20	190	19	90.00	2.000	3.000	1.000

Table 6. Metrics for the generated visual examples.

**Greedy Weighted Spanner Visualization from Java Output: $H_k \subseteq G$ ($k=2$, $2k-1=3$)
complete n10 seed102**

$|V|=10$ $|E_G|=45$ $|E_H|=10$ $\max d_H/d_G=2.09$ $w(H)/w(\text{MST})=1.21$ edge reduction = 77.8%

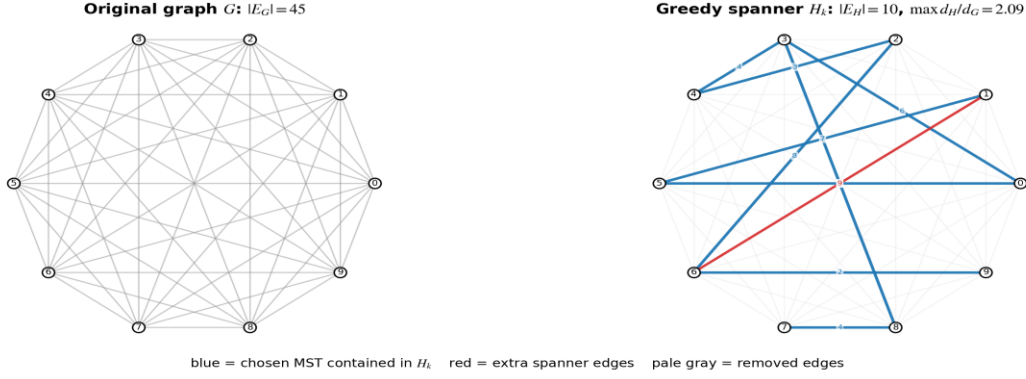


Figure 5. Complete weighted graph: 45 original edges reduced to 10 spanner edges at $k=2$.

**Greedy Weighted Spanner Visualization from Java Output: $H_k \subseteq G$ ($k=3$, $2k-1=5$)
grid 5x5 weighted seed601**

$|V|=25$ $|E_G|=40$ $|E_H|=24$ $\max d_H/d_G=4.50$ $w(H)/w(\text{MST})=1.00$ edge reduction = 40.0%

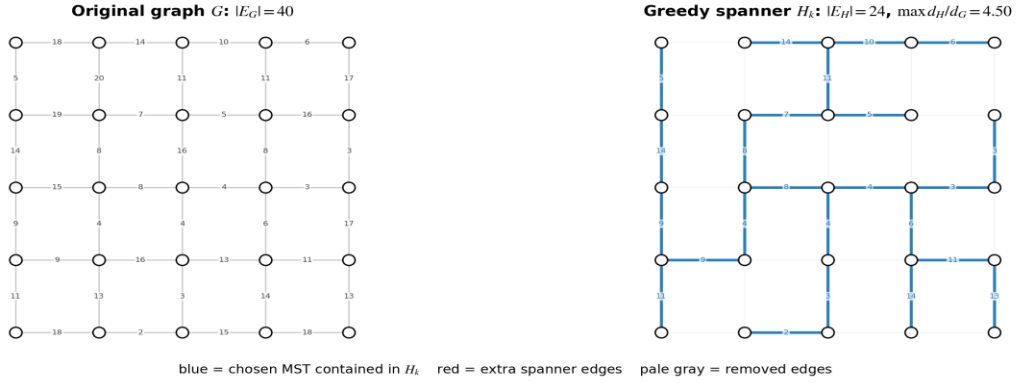


Figure 6. Weighted planar grid: 40 original edges reduced to 24 spanner edges at $k=3$.

**Greedy Weighted Spanner Visualization from Java Output: $H_k \subseteq G$ ($k=3$, $2k-1=5$)
theta lower bound targetK3 chains6**

$|V|=20$ $|E_G|=24$ $|E_H|=24$ $\max d_H/d_G=1.00$ $w(H)/w(\text{MST})=1.26$ edge reduction = 0.0%

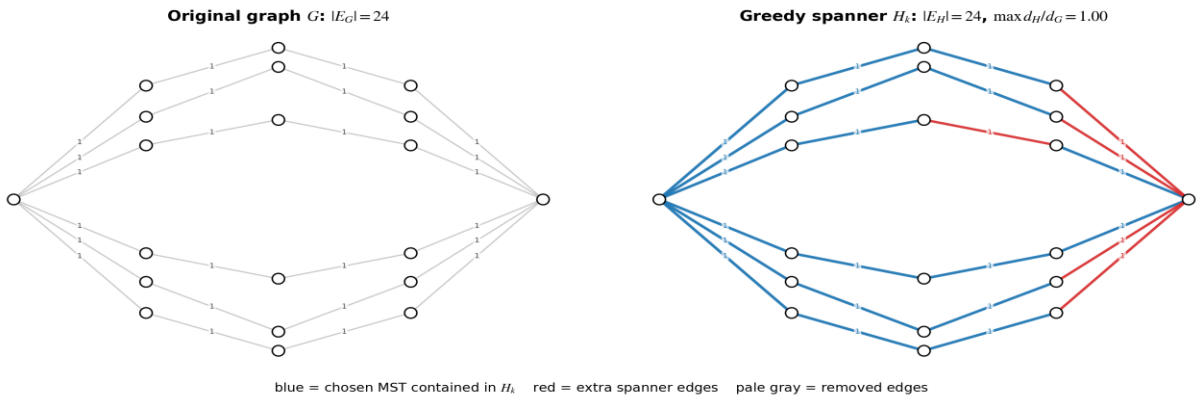


Figure 7. Theta-style example: no edge reduction at the matching target $k=3$, illustrating a hard family.

7. Discussion

7.1 What the experiments confirm

- The greedy construction is correct on the tested inputs: all measured pairs satisfy $d_H(u,v) \leq (2k-1)d_G(u,v)$.
- The sparsity/stretch tradeoff is visible: larger k gives more room to replace direct edges by paths, so fewer edges are selected.
- The weight behavior is strong in practice: for $k \geq 3$ the average lightness was close to 1, meaning the output was often near MST weight while still preserving bounded stretch.
- The graph family matters. Dense graphs shrink dramatically, while paths and trees are already minimal. Lower-bound-inspired graphs are deliberately hard to sparsify.

7.2 Interpretation of $k=1$

The $k=1$ result should not be described as "the algorithm returns the original graph". The correct statement is that $k=1$ returns a 1-spanner, meaning exact preservation of all shortest-path distances. If an original edge is not needed for any shortest path, it can be removed. This appears in the weighted complete graph results. However, in complete unit graphs, every edge has length 1 and every alternative path has length at least 2, so exact preservation forces all edges to remain.

7.3 Limitations

- The implementation prioritizes clarity over asymptotic speed. It recomputes Dijkstra during construction for each candidate edge.
- The experiment sizes are appropriate for a mini-project report, but they are not a large-scale performance benchmark.
- The planar experiments use known planar generated families; the code does not perform a general planarity test for arbitrary inputs.
- The theoretical weight bounds are worst-case guarantees; the reported lightness is empirical and should not be overclaimed as proof of a stronger theorem.

8. Conclusions and Future Work

The project satisfies the mini-project requirements: it studies a spanner paper, implements a central algorithm from that paper, performs a broad experimental evaluation, and summarizes the results. The greedy weighted spanner algorithm is simple but effective. It preserves the required stretch bound in all tested cases and often produces spanners that are both sparse and close to MST weight.

The strongest empirical pattern is that dense and weighted graphs benefit heavily from the greedy filter, while minimally connected or lower-bound-style graphs do not. This matches the theory: a spanner can only remove edges that have sufficiently short replacement paths.

Future work

- Compare against Baswana-Sen or Cohen-style faster spanner algorithms.
- Run larger benchmarks and report runtime growth separately from construction and metric evaluation.
- Add a formal planarity detector and automatically apply planar-bound comparisons only when valid.
- Add memory measurements and repeated timing runs to reduce noise.
- Compare the greedy spanner against MST, random subgraphs, and Delaunay/geometry-based spanners when inputs have coordinates.

References

- [1] I. Althofer, G. Das, D. Dobkin, D. Joseph, and J. Soares. "On Sparse Spanners of Weighted Graphs." *Discrete & Computational Geometry*, 9:81-100, 1993.
- [2] Ben-Gurion University of the Negev. "Mini-Project on Embeddings of Graphs" course syllabus, 202-1-4071, semester B 2026.
- [3] D. Peleg and A. Schaffer. "Graph Spanners." *Journal of Graph Theory*, 13(1):99-116, 1989.
- [4] S. Baswana and S. Sen. "A Linear Time Algorithm for Computing a $(2k-1)$ -Spanner of $O(n^{1+1/k})$ Size in Weighted Graphs." *ICALP* 2003.
- [5] JGraphT project. JGraphT core graph library, version 1.5.2, used by the implementation.

Appendix A. Reproducibility Commands

From the project root on Windows PowerShell:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned -Force; Get-ChildItem  
.\scripts -Recurse -File -Filter *.ps1 | Unblock-File  
.\scripts\java21.ps1  
.\scripts\compile.ps1  
.\scripts\test.ps1  
.\scripts\run_experiments.ps1 full results\results.csv  
.\scripts\make_plots.ps1
```

Expected sanity-test final line:

```
PASS all sanity tests (6/6)
```

Appendix B. Full Experiment Summary

The full run contains 270 rows over 54 scenarios and k values 1, 2, 3, 4, 5. Total stretch violations: 0. Maximum edge reduction: 96.00%. Maximum observed stretch: 9.00.